

Species Survival Risk Analyzer

AI-Powered Conservation Intelligence Using Gemma 4

Gemma 4 Good Hackathon • Kaggle × Google DeepMind • May 2026

Competition	Gemma 4 Good Hackathon — Kaggle × Google DeepMind
Submission Type	Solo
Track	Health / Climate / Biodiversity Impact
Model Used	google/gemma-4-4b-it (Transformers, on-platform)
Dataset	IUCN Red List — Endangered Species Dataset
Platform	Kaggle Notebook (GPU: T4)
Submission Date	May 18, 2026

1. Problem Statement

Over one million species currently face extinction risk, yet conservation resources — funding, personnel, and intervention capacity — are finite. The central challenge in global biodiversity conservation is not awareness: it is prioritization.

Conservation organizations rely on expert biologists to manually assess species risk, a process that is slow, expensive, and impossible to scale. Meanwhile, the IUCN Red List contains rich ecological trait data on tens of thousands of species that remains underutilized for real-time, automated risk triage.

The gap is clear: there is no accessible, AI-powered tool that can ingest structured ecological data and produce interpretable, actionable survival risk assessments at scale — especially in environments where cloud connectivity and expert access are limited.

2. Solution

The Species Survival Risk Analyzer is an on-platform AI system powered by Gemma 4 (google/gemma-4-4b-it) that ingests species ecological trait data from the IUCN Red List and produces structured, interpretable risk assessments with actionable conservation recommendations.

The system addresses three core requirements of the Gemma 4 Good Hackathon:

- Real-world impact: Directly supports global biodiversity conservation by enabling faster, scalable species triage.
- Privacy and sovereignty: Runs entirely on-platform within Kaggle’s environment. No external APIs. No data leaves the notebook.
- Technical credibility: Leverages Gemma 4’s instruction-following and structured reasoning capabilities for domain-specific JSON output extraction.

3. Technical Approach

3.1 Tech Stack

Layer	Tool / Technology
Language	Python 3.11
AI Model	google/gemma-4-4b-it (Gemma 4, 4B Instruct)
Model Runtime	HuggingFace Transformers + bfloat16 quantization
GPU	Kaggle T4 (CUDA)
Dataset	IUCN Red List — Endangered Species (Kaggle)
Inference Mode	Direct tokenizer + model.generate() — no pipeline abstraction
Visualization	Matplotlib (risk score bar chart + distribution pie chart)
Output Format	Structured JSON per species + aggregated DataFrame

3.2 Model Integration

Gemma 4 is loaded in bfloat16 precision with automatic device mapping to maximize GPU utilization on Kaggle’s T4. The model is called directly via tokenizer and model.generate() rather than through the pipeline abstraction, giving precise control over:

- Token generation length (`max_new_tokens=150`)
- Decoding strategy (`greedy, do_sample=False`) for deterministic JSON output
- New token isolation: only tokens generated beyond the prompt are decoded, eliminating prompt bleed in responses

Each species record is passed to Gemma 4 with a structured prompt that specifies the exact JSON schema required. The model returns a risk assessment containing four fields:

- `survival_risk`: categorical classification (low / medium / high / critical)
- `risk_score`: numerical score from 0–100
- `primary_threat`: natural language identification of the dominant extinction driver
- `recommendation`: one concrete, actionable conservation step

A robust JSON parsing layer handles edge cases where Gemma 4 wraps output in markdown code fences, with a structured fallback that prevents pipeline failure on parse errors.

3.3 Key Technical Decisions

Why direct `model.generate()` over pipeline?

The HuggingFace pipeline abstraction introduces conflicting `generation_config` parameters that cause undefined behavior when `max_new_tokens` is specified. Direct tokenizer + `model.generate()` eliminates this entirely and gives full reproducibility.

Why greedy decoding for JSON output?

Sampling-based decoding (`temperature > 0`) introduces stochasticity that increases JSON malformation rates. For structured output extraction, deterministic greedy decoding produces more consistent, parseable responses without sacrificing output quality at this task complexity level.

Why `bfloat16` over `float32`?

`bfloat16` halves VRAM consumption while preserving the dynamic range critical for transformer attention mechanisms — unlike `float16`, which risks numerical underflow in deeper layers. This enables the full 4B parameter model to run on a T4 without quantization artifacts.

4. Results

The system successfully analyzed 10 species from the IUCN Red List dataset, producing structured risk assessments for each. Gemma 4 demonstrated strong instruction-following behavior, returning valid JSON in all 10 inferences with ecologically coherent threat identification and risk scoring.

Key observations:

- Gemma 4 correctly identified habitat loss, climate change, and poaching as primary threats across relevant species — consistent with established IUCN threat classifications.
- Risk scores showed meaningful variance across species (not clustered), indicating the model is reasoning over trait differences rather than producing uniform outputs.
- Recommendations were specific and actionable (e.g., “establish protected corridors in primary habitat zones”) rather than generic.
- Zero pipeline failures across all 10 inferences. JSON parse fallback was not triggered.

Visualizations produced:

- Risk score bar chart: color-coded by severity threshold (green < 40, orange 40–70, red > 70)
- Risk level distribution pie chart: proportional breakdown across low / medium / high / critical classifications

5. Real-World Impact

This system directly addresses the hackathon’s core mandate: building AI applications that solve real problems in domains where expertise is scarce and resources are constrained.

Conservation organizations in low-resource environments — NGOs, field research stations, government wildlife departments in developing nations — lack access to expert biologists for real-time species triage. This tool enables:

- Rapid prioritization: Triage hundreds of species in minutes rather than weeks of expert review.
- Democratized expertise: Field workers without advanced biology training can generate defensible risk assessments using structured ecological data.
- Offline capability: With Gemma 4’s edge-optimized variants (E2B, E4B), this system can be adapted to run fully offline on low-connectivity field devices — a direct extension of the current prototype.

- Scalability: The architecture scales linearly. The same pipeline applied to the full IUCN Red List (150,000+ species) requires only compute time, not additional expertise.

6. Future Work

- Batched inference: Process full IUCN dataset in parallel batches to reduce total analysis time by 10x.
- Multimodal extension: Integrate habitat satellite imagery as additional input to Gemma 4's vision capabilities for richer risk context.
- Fine-tuning: Fine-tune Gemma 4 on IUCN expert assessments using Unsloth to improve domain-specific accuracy.
- Offline deployment: Package with Gemma 4 E4B for fully offline field deployment on conservation worker devices.
- API layer: Wrap in a FastAPI service for integration with existing conservation management platforms.

7. Conclusion

The Species Survival Risk Analyzer demonstrates that Gemma 4's instruction-following and structured reasoning capabilities can be applied directly to real conservation challenges — producing interpretable, actionable outputs from ecological data entirely on-platform, with no external dependencies.

The system is not a demo. It is a functional prototype that solves a real prioritization problem faced by conservation organizations globally. With the architectural foundation established in this submission, scaling to production deployment is an engineering problem, not a research one.

Built with Gemma 4 for the Gemma 4 Good Hackathon — Kaggle × Google DeepMind, May 2026.